Phase 1: JavaScript 핵심 (Day 01~05)

웹 풀스택 PBL 과정 - JavaScript 기초부터 DOM까지

이 교재는 프로그래밍을 처음 접하는 분들을 위해 작성되었습니다. 모든 개념을 실생활 비유와 함께 차근차근 설명합니다.

학습 로드맵

Day	주제	핵심 키워드
01	변수, 타입, 문자열, 배열	let, const, string, array
02	함수와 스코프	function, arrow, closure
03	객체와 클래스	object, class, destructuring
04	비동기 프로그래밍	Promise, async/await, fetch
05	DOM과 이벤트	DOM, event, delegation

Day 01 - 변수, 타입, 문자열, 배열 (첫 번째 할일 앱)

1.1 프로그래밍이란 무엇인가?

실생활 비유: 요리 레시피

프로그래밍은 **요리 레시피를 작성하는 것**과 같습니다.

여러분이 김치찌개를 만든다고 생각해보세요:

1. 재료를 준비한다 (변수 선언)
2. 돼지고기를 볶는다 (데이터 처리)
3. 물을 넣고 끓인다 (연산 수행)
4. 김치를 넣는다 (데이터 추가)
5. 간을 본다 (조건 확인)
6. 부족하면 소금을 더 넣는다 (조건 분기)

프로그래밍도 마찬가지입니다. **컴퓨터에게 일을 시키는 순서를 적어놓은 것**이 바로 프로그램입니다.

```
// 김치찌개 레시피를 프로그래밍으로 표현하면?
let 재료 = ["돼지고기", "김치", "두부", "파"];  // 재료를 준비한다
let 물양 = 500;                                  // 물 500ml
let 간맞음 = false;                               // 아직 간을 안 봤다

// 재료를 하나씩 넣는다
재료.forEach(function(item) {
  console.log(item + "을(를) 넣습니다");
});

// 간을 확인한다
if (!간맞음) {
  console.log("소금을 더 넣습니다");
}
```

왜 필요한가?

우리가 매일 쓰는 **카카오톡**, **배달의민족**, **인스타그램** 모두 프로그래밍으로 만들어졌습니다. 프로그래밍을 배우면 여러분도 이런 서비스를 직접 만들 수 있습니다.

1.2 JavaScript를 배우는 이유

- **웹 브라우저에서 바로 실행**됩니다 (별도의 복잡한 설치 불필요)
- **프론트엔드**(화면)와 **백엔드**(서버) 모두 만들 수 있습니다
- **세계에서 가장 많이 쓰이는 언어** 중 하나입니다
- 취업 시장에서 수요가 매우 높습니다

1.3 개발 환경 설정

Node.js 설치

Node.js는 JavaScript를 브라우저 밖에서도 실행할 수 있게 해주는 도구입니다.

1. https://nodejs.org 접속
2. **LTS 버전** 다운로드 (안정적인 버전)
3. 설치 후 터미널에서 확인:

```
# 터미널(맥) 또는 명령 프롬프트(윈도우)에서 입력
node --version    # v20.x.x 같은 버전이 나오면 성공!
npm --version     # npm도 함께 설치됩니다
```

VS Code 설치

1. https://code.visualstudio.com 에서 다운로드
2. 추천 확장 프로그램:
   - **Korean Language Pack** (한국어 지원)
   - **ESLint** (코드 오류 검사)
   - **Prettier** (코드 자동 정리)

첫 번째 코드 실행하기

```
// hello.js 파일을 만들고 아래 내용을 작성하세요
console.log("안녕하세요! 첫 번째 JavaScript 프로그램입니다!");
```

```
# 터미널에서 실행
node hello.js
# 결과: 안녕하세요! 첫 번째 JavaScript 프로그램입니다!
```

1.4 변수 - 데이터를 담는 상자

실생활 비유: 이름표가 붙은 상자

변수는 **이름표가 붙은 상자**입니다. 상자 안에 물건(데이터)을 넣고, 이름표(변수명)를 보고 꺼내 쓸 수 있습니다.

```
// 변수 선언: 상자를 만들고 이름표를 붙인다
let 이름 = "김철수";         // "이름"이라는 상자에 "김철수"를 넣음
let 나이 = 25;               // "나이"라는 상자에 25를 넣음
let 학생인가 = true;         // "학생인가"라는 상자에 true(맞다)를 넣음

// 변수 사용: 상자에서 꺼내 쓴다
console.log(이름);           // "김철수" 출력
console.log(나이);           // 25 출력
```

let vs const: 우편함 vs 금고

```
// let: 우편함 - 내용물을 바꿀 수 있다
let score = 0;          // 처음에 0점
score = 100;            // 100점으로 변경 가능!
console.log(score);     // 100

// const: 금고 - 한번 넣으면 바꿀 수 없다
const birthYear = 1999;    // 태어난 해는 변하지 않는다
// birthYear = 2000;       // 에러! const는 변경 불가!

// const: 생년월일, 이름, 주민번호처럼 변하지 않는 값에 사용
const PI = 3.14159;        // 원주율은 절대 변하지 않는다
const APP_NAME = "할일관리";  // 앱 이름도 고정
```

> **팁:** 기본적으로 `const` 를 쓰고, 값이 바뀌어야 할 때만 `let` 을 쓰세요. 이것이 현대 JavaScript의 모범 사례입니다.

var를 쓰지 않는 이유

```
// var의 문제점 1: 같은 이름으로 다시 선언해도 에러가 나지 않는다
var name = "철수";
var name = "영희";     // 에러 없음! 실수를 발견하기 어렵다

// let이라면?
let name2 = "철수";
// let name2 = "영희";  // 에러 발생! 실수를 바로 잡을 수 있다

// var의 문제점 2: 블록 스코프를 무시한다 (Day 02에서 자세히 설명)
if (true) {
  var leaked = "나는 밖에서도 보여요";
  let blocked = "나는 이 안에서만 보여요";
}
console.log(leaked);    // "나는 밖에서도 보여요" (이상한 동작!)
// console.log(blocked); // 에러! (이게 정상적인 동작)
```

자주 하는 실수

```
// 실수 1: 선언 없이 변수 사용
// console.log(myVar);  // 에러! myVar가 선언되지 않았음

// 실수 2: const 변수에 재할당
const MAX = 100;
// MAX = 200;             // 에러! const는 재할당 불가

// 실수 3: 변수명에 공백이나 특수문자 사용
// let my name = "철수";     // 에러! 공백 불가
// let 123abc = "test";      // 에러! 숫자로 시작 불가
let myName = "철수";         // 이렇게 카멜케이스(camelCase) 사용
let my_name = "철수";        // 또는 스네이크케이스(snake_case) 사용
```

1.5 데이터 타입 - 상자에 넣을 수 있는 것들

실생활 비유: 편의점 상품 분류

편의점에서 음료, 과자, 도시락을 각각 다른 선반에 놓듯이, JavaScript에서도 데이터를 **종류별로 구분**합니다.

```
// 1. number (숫자) - 가격표처럼 계산할 수 있는 값
let price = 3500;           // 정수
let temperature = 36.5;     // 소수
let negative = -10;         // 음수

// 2. string (문자열) - 카카오톡 메시지처럼 글자로 된 값
let message = "안녕하세요";      // 큰따옴표
let name = '김철수';             // 작은따옴표 (둘 다 같음)
let greeting = `반갑습니다`;     // 백틱 (특별한 기능이 있음, 뒤에서 설명)

// 3. boolean (참/거짓) - 스위치 ON/OFF 같은 값
let isLoggedIn = true;      // 로그인 했다 (ON)
let isAdmin = false;        // 관리자가 아니다 (OFF)

// 4. null - 일부러 "비어있음"으로 표시한 값
// 배달앱에서 "배달 완료"가 되면 배달기사 정보를 null로 바꾸는 것과 비슷
let selectedItem = null;    // 아직 아무것도 선택하지 않았음을 명시

// 5. undefined - 아직 값이 정해지지 않은 상태
// 시험지를 받았는데 아직 이름을 안 쓴 것과 비슷
let futureValue;            // 값을 안 넣으면 자동으로 undefined
console.log(futureValue);   // undefined
```

typeof로 타입 확인하기

```
// typeof 연산자: "이 상자 안에 뭐가 들어있지?" 라고 물어보는 것
console.log(typeof 42);           // "number"
console.log(typeof "안녕");       // "string"
console.log(typeof true);         // "boolean"
console.log(typeof undefined);    // "undefined"
console.log(typeof null);         // "object" (JavaScript의 오래된 버그!)
console.log(typeof [1, 2, 3]);    // "object" (배열도 객체의 일종)
```

타입 변환 - 자동 vs 수동

```
// 자동 변환 (위험할 수 있음!)
console.log("5" + 3);      // "53" (문자열 연결! 숫자 8이 아님!)
console.log("5" - 3);      // 2   (빼기는 숫자로 자동 변환)
console.log("5" * 2);      // 10  (곱하기도 숫자로 자동 변환)

// 수동 변환 (안전한 방법)
let strNumber = "42";
let realNumber = Number(strNumber);    // 문자열 → 숫자
console.log(realNumber + 8);           // 50 (정확한 계산!)

let num = 100;
let strNum = String(num);     // 숫자 → 문자열
console.log(typeof strNum);   // "string"

// 문자열을 숫자로 바꾸는 다른 방법
let parsed = parseInt("42px");     // 42 (앞의 숫자만 추출)
let floated = parseFloat("3.14"); // 3.14 (소수점 포함 추출)
```

1.6 문자열 다루기

왜 필요한가?

회원가입 폼에서 사용자가 입력한 이름 앞뒤 공백 제거, 이메일 형식 확인, 채팅 메시지 표시 등 **웹 개발에서 문자열은 가장 많이 다루는 데이터**입니다.

템플릿 리터럴 (백틱 문자열)

```javascript
// 과거 방식: 문자열 연결이 불편
let name = "김철수";
let age = 25;
let old = "저는 " + name + "이고, " + age + "살입니다.";  // 복잡!

// 현대 방식: 템플릿 리터럴 (백틱 ` 사용)
let modern = `저는 ${name}이고, ${age}살입니다.`;  // 깔끔!
console.log(modern);  // "저는 김철수이고, 25살입니다."

// ${} 안에서 계산도 가능
console.log(`내년이면 ${age + 1}살이에요`);  // "내년이면 26살이에요"

// 여러 줄 문자열도 가능
let multiLine = `
  첫 번째 줄
  두 번째 줄
  세 번째 줄
`;
```

주요 문자열 메서드

```javascript
let text = "  Hello, JavaScript World!  ";

// length: 문자열 길이 (글자 수)
console.log(text.length);           // 29 (공백 포함)

// trim(): 앞뒤 공백 제거 (회원가입 폼에서 필수!)
let trimmed = text.trim();
console.log(trimmed);               // "Hello, JavaScript World!"

// toUpperCase() / toLowerCase(): 대소문자 변환
console.log("hello".toUpperCase()); // "HELLO"
console.log("HELLO".toLowerCase()); // "hello"

// includes(): 특정 문자가 포함되어 있는지 확인
let email = "user@example.com";
console.log(email.includes("@"));   // true (이메일에 @가 있다)
console.log(email.includes("abc")); // false

// slice(시작, 끝): 문자열 자르기
let greeting = "안녕하세요, 반갑습니다";
console.log(greeting.slice(0, 5));  // "안녕하세요" (0번째부터 4번째까지)
console.log(greeting.slice(7));     // "반갑습니다" (7번째부터 끝까지)

// split(구분자): 문자열을 배열로 쪼개기
let csv = "사과,바나나,딸기,포도";
let fruits = csv.split(",");        // ["사과", "바나나", "딸기", "포도"]
console.log(fruits[0]);             // "사과"

// replace(): 문자열 치환
let msg = "나는 고양이를 좋아합니다";
let newMsg = msg.replace("고양이", "강아지");
console.log(newMsg);                // "나는 강아지를 좋아합니다"

// indexOf(): 특정 문자의 위치 찾기
console.log("JavaScript".indexOf("Script"));  // 4 (4번째 위치)
console.log("JavaScript".indexOf("Python"));  // -1 (없으면 -1)
```

실습: 이름 포맷팅

```
// 사용자가 입력한 이름을 깔끔하게 정리하는 코드
let rawInput = "   kim chul su   ";

// 1단계: 앞뒤 공백 제거
let cleaned = rawInput.trim();
console.log(cleaned);  // "kim chul su"

// 2단계: 공백으로 쪼개기
let parts = cleaned.split(" ");
console.log(parts);    // ["kim", "chul", "su"]

// 3단계: 각 단어의 첫 글자를 대문자로
let formatted = parts.map(function(word) {
  // 첫 글자를 대문자로 + 나머지 글자
  return word[0].toUpperCase() + word.slice(1).toLowerCase();
});
console.log(formatted);  // ["Kim", "Chul", "Su"]

// 4단계: 다시 합치기
let fullName = formatted.join(" ");
console.log(fullName);   // "Kim Chul Su"
```

1.7 배열 - 여러 데이터를 순서대로 담기

실생활 비유: 서랍장

배열은 **번호가 매겨진 서랍장**입니다. 각 서랍(인덱스)에 하나씩 물건(데이터)을 넣을 수 있습니다.

```
// 배열 만들기: 서랍장을 준비하고 물건을 넣는다
let fruits = ["사과", "바나나", "딸기", "포도"];

// 인덱스: 서랍 번호는 0부터 시작한다! (주의!)
//          0번      1번       2번      3번
console.log(fruits[0]);    // "사과"   (첫 번째)
console.log(fruits[1]);    // "바나나" (두 번째)
console.log(fruits[3]);    // "포도"   (네 번째)
console.log(fruits.length); // 4 (총 4개)
```

배열 기본 조작: push, pop, shift, unshift

```javascript
let cart = ["우유", "빵"];
console.log(cart);  // ["우유", "빵"]

// push: 맨 뒤에 추가 (편의점 계산대에 줄 서기)
cart.push("계란");
console.log(cart);  // ["우유", "빵", "계란"]

// pop: 맨 뒤에서 제거 (줄의 마지막 사람이 나감)
let removed = cart.pop();
console.log(removed);  // "계란"
console.log(cart);     // ["우유", "빵"]

// unshift: 맨 앞에 추가 (새치기!)
cart.unshift("콜라");
console.log(cart);  // ["콜라", "우유", "빵"]

// shift: 맨 앞에서 제거 (첫 번째 사람이 계산 완료)
let first = cart.shift();
console.log(first);  // "콜라"
console.log(cart);   // ["우유", "빵"]
```

배열 반복 메서드: forEach, map, filter, find

```
let numbers = [1, 2, 3, 4, 5];

// forEach: 하나씩 꺼내서 뭔가 하기 (택배 기사가 하나씩 배달)
numbers.forEach(function(num) {
  console.log(`현재 숫자: ${num}`);
});
// 현재 숫자: 1
// 현재 숫자: 2
// ... (하나씩 출력)

// map: 하나씩 변환해서 새 배열 만들기 (공장 컨베이어 벨트)
// 각 숫자를 2배로 만든 새 배열
let doubled = numbers.map(function(num) {
  return num * 2;
});
console.log(doubled);   // [2, 4, 6, 8, 10]
console.log(numbers);   // [1, 2, 3, 4, 5] (원본은 그대로!)

// filter: 조건에 맞는 것만 골라내기 (면접에서 합격자만 선발)
// 3보다 큰 숫자만 골라내기
let bigNumbers = numbers.filter(function(num) {
  return num > 3;
});
console.log(bigNumbers);  // [4, 5]

// find: 조건에 맞는 첫 번째 요소 찾기 (분실물 센터)
let found = numbers.find(function(num) {
  return num > 3;
});
console.log(found);  // 4 (조건에 맞는 첫 번째만 반환)
```

실습: 할일 목록 만들기

```javascript
// === 간단한 할일 목록 프로그램 ===

// 1. 할일 배열 만들기
let todos = ["장보기", "운동하기", "공부하기", "빨래하기"];
console.log("전체 할일:", todos);
// 전체 할일: ["장보기", "운동하기", "공부하기", "빨래하기"]

// 2. 새 할일 추가
todos.push("요리하기");
console.log("추가 후:", todos);
// 추가 후: ["장보기", "운동하기", "공부하기", "빨래하기", "요리하기"]

// 3. 첫 번째 할일 완료 (제거)
let done = todos.shift();
console.log(`"${done}" 완료!`);
// "장보기" 완료!

// 4. "하기"가 포함된 할일 필터링
let withHagi = todos.filter(function(todo) {
  return todo.includes("하기");
});
console.log("'하기' 포함:", withHagi);
// '하기' 포함: ["운동하기", "공부하기", "빨래하기", "요리하기"]

// 5. 각 할일에 번호 붙이기
let numbered = todos.map(function(todo, index) {
  return `${index + 1}. ${todo}`;
});
console.log("번호 붙인 목록:", numbered);
// 번호 붙인 목록: ["1. 운동하기", "2. 공부하기", "3. 빨래하기", "4. 요리하기"]

// 6. "공부하기" 찾기
let studyItem = todos.find(function(todo) {
  return todo === "공부하기";
});
console.log("찾은 항목:", studyItem);  // "공부하기"
```

console.log 디버깅 팁

```javascript
// 코드가 예상대로 동작하지 않을 때, console.log로 중간 값을 확인하세요
let data = [10, 20, 30];
console.log("원본 데이터:", data);            // 원본 확인

let result = data.map(function(x) {
  console.log("현재 처리 중:", x);            // 각 단계 확인
  return x * 2;
});
console.log("최종 결과:", result);            // 결과 확인

// 여러 값을 한번에 확인
let name = "철수";
let age = 25;
console.log("이름:", name, "나이:", age);     // 이름: 철수 나이: 25

// 객체/배열을 보기 좋게 출력
console.table(["사과", "바나나", "딸기"]);    // 표 형태로 출력
```

실습 파일 안내: `examples/phase1-js/day01-variables/` 폴더에서 실습 파일을 확인하세요.

Day 02 - 함수와 스코프 (코드 재사용하기)

2.1 함수란?

실생활 비유: 자판기

함수는 **자판기**와 같습니다.

1. **동전을 넣는다** (입력 = 매개변수)
2. **버튼을 누른다** (함수 호출)
3. **음료가 나온다** (출력 = 반환값)

자판기가 있으면 매번 음료를 직접 만들 필요가 없듯이, 함수가 있으면 **같은 코드를 반복해서 쓸 필요가 없습니다**.

왜 필요한가?

```
// 함수 없이 인사하기 - 같은 코드를 계속 반복해야 한다!
console.log("안녕하세요, 철수님! 오늘도 좋은 하루 되세요!");
console.log("안녕하세요, 영희님! 오늘도 좋은 하루 되세요!");
console.log("안녕하세요, 민수님! 오늘도 좋은 하루 되세요!");

// 함수로 만들면 - 한 번만 작성하고 계속 재사용!
function greet(name) {
  console.log(`안녕하세요, ${name}님! 오늘도 좋은 하루 되세요!`);
}
greet("철수");  // 안녕하세요, 철수님! 오늘도 좋은 하루 되세요!
greet("영희");  // 안녕하세요, 영희님! 오늘도 좋은 하루 되세요!
greet("민수");  // 안녕하세요, 민수님! 오늘도 좋은 하루 되세요!
```

2.2 함수 선언 방식 세 가지

함수 선언식 (Function Declaration)

```
// 가장 기본적인 방식
// function 키워드 + 함수이름 + (매개변수) + { 본문 }
function add(a, b) {
  return a + b;  // return: 결과를 돌려준다 (자판기에서 음료가 나오는 것)
}

let result = add(3, 5);  // 함수 호출: 3과 5를 넣고 결과를 받는다
console.log(result);      // 8
```

함수 표현식 (Function Expression)

```
// 함수를 변수에 담는 방식
const multiply = function(a, b) {
  return a * b;
};

console.log(multiply(4, 5));  // 20
```

화살표 함수 (Arrow Function) - 현대 JavaScript의 대세

```
// 화살표 함수: 더 짧고 간결하게 작성
const subtract = (a, b) => {
  return a - b;
};

// 본문이 한 줄이면 중괄호와 return 생략 가능!
const divide = (a, b) => a / b;

console.log(subtract(10, 3));  // 7
console.log(divide(10, 2));    // 5

// 매개변수가 하나면 괄호도 생략 가능
const double = n => n * 2;
console.log(double(5));  // 10
```

세 가지 비교

```
// 선언식: 전통적인 방식, 호이스팅(먼저 사용 가능) 됨
function sayHello1(name) {
  return `안녕, ${name}!`;
}

// 표현식: 변수에 할당, 호이스팅 안 됨
const sayHello2 = function(name) {
  return `안녕, ${name}!`;
};

// 화살표: 가장 간결, 콜백 함수에서 많이 사용
const sayHello3 = (name) => `안녕, ${name}!`;

// 세 함수 모두 같은 결과
console.log(sayHello1("철수"));  // "안녕, 철수!"
console.log(sayHello2("철수"));  // "안녕, 철수!"
console.log(sayHello3("철수"));  // "안녕, 철수!"
```

> **실무 팁:** 일반 함수는 `function` 선언식, 콜백(다른 함수에 넘기는 함수)은 화살표 함수를 주로 사용합니다.

2.3 매개변수와 반환값

기본값 매개변수

```
// 배달앱에서 배달비 기본값이 3000원인 것처럼
function calculateTotal(price, deliveryFee = 3000) {
  return price + deliveryFee;
}

console.log(calculateTotal(15000));         // 18000 (배달비 기본값 3000)
console.log(calculateTotal(15000, 5000));   // 20000 (배달비 5000으로 변경)
console.log(calculateTotal(15000, 0));      // 15000 (배달비 무료!)
```

나머지 매개변수 (...args)

```
// 편의점에서 물건을 몇 개든 계산할 수 있는 것처럼
function calculateSum(...numbers) {
  // numbers는 배열로 들어온다
  let total = 0;
  for (let num of numbers) {
    total += num;  // total = total + num
  }
  return total;
}

console.log(calculateSum(1, 2, 3));         // 6
console.log(calculateSum(100, 200));        // 300
console.log(calculateSum(10, 20, 30, 40)); // 100
```

2.4 콜백 함수

실생활 비유: 배달 완료 후 알림

콜백 함수는 "이 일이 끝나면 이걸 해줘"라고 부탁하는 것입니다. 배달의민족에서 "배달 완료되면 알림 보내줘"라고 설정하는 것과 같습니다.

```
// 콜백 함수: 다른 함수에 "넘겨주는" 함수
function processOrder(menu, callback) {
  console.log(`${menu} 주문 접수!`);
  console.log(`${menu} 조리 중...`);
  console.log(`${menu} 조리 완료!`);
  callback(menu);  // 조리가 끝나면 콜백 함수를 실행
}

// "배달하기" 함수를 콜백으로 전달
function deliver(menu) {
  console.log(`${menu} 배달 출발!`);
}

processOrder("치킨", deliver);
// 치킨 주문 접수!
// 치킨 조리 중...
// 치킨 조리 완료!
// 치킨 배달 출발!
```

배열 메서드와 콜백 함수

```
let scores = [85, 92, 78, 96, 64, 88];

// map + 화살표 함수 콜백: 각 점수에 5점 추가
let curved = scores.map(score => score + 5);
console.log(curved);  // [90, 97, 83, 101, 69, 93]

// filter + 화살표 함수 콜백: 80점 이상만 골라내기
let passed = scores.filter(score => score >= 80);
console.log(passed);  // [85, 92, 96, 88]

// reduce: 모든 값을 하나로 합치기 (총합 구하기)
let total = scores.reduce((sum, score) => sum + score, 0);
//                          누적값  현재값              초기값
console.log(total);    // 503
console.log(total / scores.length);  // 약 83.8 (평균)

// sort: 정렬 (비교 함수 콜백 필요!)
let sorted = [...scores].sort((a, b) => a - b);  // 오름차순
console.log(sorted);   // [64, 78, 85, 88, 92, 96]

let descending = [...scores].sort((a, b) => b - a);  // 내림차순
console.log(descending);  // [96, 92, 88, 85, 78, 64]
```

자주 하는 실수

```
// 실수: sort()를 비교 함수 없이 사용하면 문자열로 정렬됨!
let nums = [10, 9, 2, 100, 30];
console.log(nums.sort());           // [10, 100, 2, 30, 9] (문자열 순서!)
console.log(nums.sort((a, b) => a - b));  // [2, 9, 10, 30, 100] (숫자 순서!)
```

2.5 스코프 (Scope) - 변수의 사용 범위

실생활 비유: 건물의 층

스코프는 **건물의 층**과 같습니다.

- **전역 스코프**: 1층 로비 - 누구나 접근 가능
- **함수 스코프**: 각 사무실 - 그 사무실 사람만 접근
- **블록 스코프**: 사무실 안의 개인 서랍 - 그 자리 사람만 접근

```
// 전역 스코프: 어디서든 접근 가능 (1층 로비의 안내판)
const APP_NAME = "할일관리앱";

function showInfo() {
  // 함수 스코프: 이 함수 안에서만 접근 가능 (사무실 내부)
  let version = "1.0.0";
  console.log(APP_NAME);    // 전역 변수 접근 가능 (로비는 누구나)
  console.log(version);      // 함수 내부 변수 접근 가능

  if (true) {
    // 블록 스코프: 이 if 블록 안에서만 접근 가능 (개인 서랍)
    let secret = "비밀번호";
    console.log(secret);     // 블록 안에서 접근 가능
    console.log(version);    // 바깥 함수 변수도 접근 가능
  }

  // console.log(secret);  // 에러! 블록 밖에서는 접근 불가
}

showInfo();
// console.log(version);   // 에러! 함수 밖에서는 접근 불가
```

2.6 클로저 (Closure)

실생활 비유: 배낭

클로저는 **배낭을 메고 여행하는 것**과 같습니다. 함수가 만들어진 곳의 변수들을 "배낭"에 넣고 다니면서, 나중에 어디서든 꺼내 쓸 수 있습니다.

```javascript
// 클로저 예제: 카운터 만들기
function createCounter() {
  let count = 0;  // 이 변수가 "배낭" 안에 들어간다

  return {
    // increase 함수는 count를 "기억"한다 (배낭에서 꺼내 쓴다)
    increase() {
      count += 1;
      return count;
    },
    decrease() {
      count -= 1;
      return count;
    },
    getCount() {
      return count;
    }
  };
}

const counter = createCounter();
console.log(counter.increase());  // 1
console.log(counter.increase());  // 2
console.log(counter.increase());  // 3
console.log(counter.decrease());  // 2
console.log(counter.getCount());  // 2

// 새로운 카운터는 독립적 (별도의 배낭)
const counter2 = createCounter();
console.log(counter2.increase());  // 1 (counter와 별개!)
```

클로저의 실용적 사용

```
// 실용 예제: 할인 계산기
function createDiscountCalculator(discountRate) {
  // discountRate가 클로저에 저장된다
  return function(price) {
    const discount = price * (discountRate / 100);
    return price - discount;
  };
}

// VIP 고객용 20% 할인 계산기
const vipDiscount = createDiscountCalculator(20);
// 일반 고객용 10% 할인 계산기
const normalDiscount = createDiscountCalculator(10);

console.log(vipDiscount(10000));    // 8000 (20% 할인)
console.log(normalDiscount(10000)); // 9000 (10% 할인)
console.log(vipDiscount(50000));    // 40000 (20% 할인)
```

2.7 IIFE (즉시 실행 함수)

```
// IIFE: 만들자마자 바로 실행되는 함수
// 전역 변수 오염을 방지하기 위해 사용

(function() {
  let privateVar = "외부에서 접근 불가";
  console.log(privateVar);  // "외부에서 접근 불가"
})();

// console.log(privateVar);  // 에러! 밖에서 접근 불가

// 실용 예제: 초기 설정
const config = (function() {
  const API_URL = "https://api.example.com";
  const VERSION = "2.0";

  return {
    getApiUrl: () => API_URL,
    getVersion: () => VERSION
  };
})();

console.log(config.getApiUrl());   // "https://api.example.com"
console.log(config.getVersion());  // "2.0"
```

2.8 실습: 할일 필터링/정렬 함수

```javascript
// === 할일 목록 필터링 및 정렬 시스템 ===

// 할일 데이터
const todos = [
  { id: 1, text: "장보기", priority: "high", done: false },
  { id: 2, text: "운동하기", priority: "medium", done: true },
  { id: 3, text: "공부하기", priority: "high", done: false },
  { id: 4, text: "청소하기", priority: "low", done: false },
  { id: 5, text: "빨래하기", priority: "medium", done: true },
];

// 1. 완료되지 않은 할일만 필터링
const pending = todos.filter(todo => !todo.done);
console.log("미완료:", pending.map(t => t.text));
// 미완료: ["장보기", "공부하기", "청소하기"]

// 2. 우선순위별 필터링 함수 (클로저 활용!)
function createPriorityFilter(priority) {
  return function(todoList) {
    return todoList.filter(todo => todo.priority === priority);
  };
}

const getHighPriority = createPriorityFilter("high");
const getMediumPriority = createPriorityFilter("medium");

console.log("높은 우선순위:", getHighPriority(todos).map(t => t.text));
// 높은 우선순위: ["장보기", "공부하기"]

// 3. 우선순위 순으로 정렬
const priorityOrder = { high: 1, medium: 2, low: 3 };
const sorted = [...todos].sort((a, b) => {
  return priorityOrder[a.priority] - priorityOrder[b.priority];
});
console.log("정렬된 할일:", sorted.map(t => `[${t.priority}] ${t.text}`));
// 정렬된 할일: ["[high] 장보기", "[high] 공부하기", "[medium] 운동하기", ...]

// 4. 통계 함수
function getTodoStats(todoList) {
  const total = todoList.length;
  const completed = todoList.filter(t => t.done).length;
  const pending = total - completed;

  return {
    total,
    completed,
    pending,
    completionRate: `${Math.round((completed / total) * 100)}%`
  };
}
```

```
console.log(getTodoStats(todos));
// { total: 5, completed: 2, pending: 3, completionRate: "40%" }
```

Day 03 - 객체와 클래스 (데이터 구조화)

3.1 객체란?

실생활 비유: 신분증

객체는 **신분증**과 같습니다. 하나의 카드에 이름, 나이, 주소, 사진 등 **관련 정보를 모아놓은 것**입니다.

```
// 배열로 사람 정보를 관리하면? → 불편하고 헷갈린다
let person1 = ["김철수", 25, "서울시 강남구", "010-1234-5678"];
// person1[0]이 이름인지, 나이인지 알기 어렵다!

// 객체로 관리하면? → 깔끔하고 직관적!
let person = {
  name: "김철수",            // 이름
  age: 25,                   // 나이
  address: "서울시 강남구",  // 주소
  phone: "010-1234-5678"     // 전화번호
};

// 이름표(키)를 보고 바로 데이터를 찾을 수 있다
console.log(person.name);     // "김철수" (점 표기법)
console.log(person["age"]);   // 25 (대괄호 표기법)
```

왜 필요한가?

```
// 실제 웹 앱에서 사용자 정보를 다루는 예
const user = {
  id: 1,
  username: "chulsu_kim",
  email: "chulsu@example.com",
  isVerified: true,
  posts: 42,
  followers: 128
};

// 이렇게 관련 데이터를 하나로 묶으면 관리가 편하다
console.log(`${user.username}님의 게시글: ${user.posts}개`);
// "chulsu_kim님의 게시글: 42개"
```

3.2 객체 리터럴 { }

프로퍼티 접근

```
const product = {
  name: "아이폰 15",
  price: 1250000,
  color: "블루",
  inStock: true
};

// 점 표기법: 키 이름을 직접 쓸 때 (가장 일반적)
console.log(product.name);      // "아이폰 15"
console.log(product.price);     // 1250000

// 대괄호 표기법: 변수에 키 이름이 담겨있을 때
let key = "color";
console.log(product[key]);      // "블루"

// 프로퍼티 추가
product.brand = "Apple";        // 새 프로퍼티 추가
console.log(product.brand);    // "Apple"

// 프로퍼티 수정
product.price = 1100000;        // 가격 변경
console.log(product.price);    // 1100000

// 프로퍼티 삭제
delete product.inStock;
console.log(product.inStock);  // undefined (삭제됨)
```

메서드 (객체 안의 함수)

```
// 객체 안에 함수를 넣으면 "메서드"라고 부른다
const calculator = {
  // 메서드 정의
  add(a, b) {
    return a + b;
  },
  subtract(a, b) {
    return a - b;
  },
  // this: 이 객체 자신을 가리킨다
  describe() {
    return "이것은 계산기 객체입니다";
  }
};

console.log(calculator.add(10, 5));        // 15
console.log(calculator.subtract(10, 5));   // 5
console.log(calculator.describe());        // "이것은 계산기 객체입니다"
```

3.3 구조 분해 할당 (Destructuring)

실생활 비유: 택배 상자 열기

택배가 왔을 때 상자를 열어서 안에 있는 물건들을 하나씩 꺼내놓는 것과 같습니다.

객체 구조 분해

```
const student = {
  name: "이영희",
  age: 22,
  major: "컴퓨터공학",
  gpa: 3.8
};

// 과거 방식: 하나씩 꺼내기 (번거롭다)
// const name = student.name;
// const age = student.age;
// const major = student.major;

// 구조 분해 할당: 한 번에 꺼내기! (택배 상자를 한번에 열기)
const { name, age, major, gpa } = student;
console.log(name);   // "이영희"
console.log(major);  // "컴퓨터공학"

// 별칭(alias): 다른 이름으로 꺼내기
const { name: studentName, age: studentAge } = student;
console.log(studentName);  // "이영희"

// 기본값: 없는 값에 기본값 설정
const { name: n, scholarship = "없음" } = student;
console.log(scholarship);  // "없음" (student에 scholarship이 없으므로)
```

배열 구조 분해

```
const colors = ["빨강", "파랑", "초록", "노랑"];

// 배열 구조 분해: 순서대로 꺼내기
const [first, second, third] = colors;
console.log(first);   // "빨강"
console.log(second);  // "파랑"
console.log(third);   // "초록"

// 일부만 꺼내기: 쉼표로 건너뛰기
const [, , thirdColor] = colors;
console.log(thirdColor);  // "초록"

// 나머지 모아서 꺼내기
const [head, ...rest] = colors;
console.log(head);  // "빨강"
console.log(rest);  // ["파랑", "초록", "노랑"]

// 실용 예제: 함수 반환값 구조 분해
function getMinMax(numbers) {
  return [Math.min(...numbers), Math.max(...numbers)];
}

const [min, max] = getMinMax([3, 7, 1, 9, 4]);
console.log(`최솟값: ${min}, 최댓값: ${max}`);  // "최솟값: 1, 최댓값: 9"
```

3.4 스프레드/나머지 연산자 (...)

객체 복사와 합치기

```
// 원본 객체
const original = { name: "철수", age: 25 };

// 스프레드로 복사: 원본과 독립된 새 객체 생성
const copy = { ...original };
copy.age = 30;
console.log(original.age);  // 25 (원본은 그대로!)
console.log(copy.age);      // 30

// 객체 합치기 + 프로퍼티 추가/덮어쓰기
const updated = { ...original, age: 26, email: "cs@test.com" };
console.log(updated);
// { name: "철수", age: 26, email: "cs@test.com" }
```

배열 복사와 합치기

```
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];

// 배열 합치기
const combined = [...arr1, ...arr2];
console.log(combined);  // [1, 2, 3, 4, 5, 6]

// 배열 복사 후 요소 추가
const withNew = [...arr1, 99];
console.log(withNew);   // [1, 2, 3, 99]
console.log(arr1);      // [1, 2, 3] (원본 그대로)
```

왜 직접 수정하면 안 되는가? (불변성)

```
// 나쁜 예: 원본을 직접 수정 (다른 곳에서 원본을 쓰고 있으면 문제 발생!)
const todo = { text: "공부", done: false };
todo.done = true;  // 원본이 바뀌어 버림!

// 좋은 예: 새 객체를 만들어서 사용 (원본 보존)
const todo2 = { text: "공부", done: false };
const updatedTodo = { ...todo2, done: true };  // 새 객체 생성
console.log(todo2.done);        // false (원본 보존)
console.log(updatedTodo.done); // true  (새 객체만 변경)
```

> **왜 중요한가?** React 같은 프레임워크에서는 데이터가 변경되었는지를 "새 객체인지"로 판단합니다. 원본을 직접 수정하면 화면이 업데이트되지 않는 버그가 발생합니다.

3.5 클래스 (Class)

실생활 비유: 붕어빵 틀

클래스는 **붕어빵 틀**과 같습니다.

- **클래스**: 붕어빵 틀 (설계도)
- **인스턴스**: 틀로 만든 실제 붕어빵 (실체)
- **constructor**: 반죽과 팥을 넣는 과정 (초기화)

- **메서드**: 붕어빵으로 할 수 있는 일 (먹기, 선물하기)

```
// 클래스 정의: 붕어빵 틀 만들기
class Todo {
  // constructor: 새 할일을 만들 때 자동으로 실행
  constructor(text) {
    this.text = text;        // this = 지금 만들어지는 할일 자체
    this.done = false;       // 처음엔 미완료 상태
    this.createdAt = new Date();  // 생성 시간 자동 기록
  }

  // 메서드: 할일을 완료하기
  complete() {
    this.done = true;
    console.log(`"${this.text}" 완료!`);
  }

  // 메서드: 할일 정보 출력
  toString() {
    const status = this.done ? "[완료]" : "[미완료]";
    return `${status} ${this.text}`;
  }
}

// 인스턴스 만들기: 붕어빵 틀에 반죽 넣기
const todo1 = new Todo("장보기");
const todo2 = new Todo("운동하기");

console.log(todo1.toString());  // "[미완료] 장보기"
todo1.complete();                // "장보기" 완료!
console.log(todo1.toString());  // "[완료] 장보기"
console.log(todo2.toString());  // "[미완료] 운동하기"
```

TodoManager 클래스 예제

```javascript
class TodoManager {
  constructor() {
    this.todos = [];        // 할일 목록 배열
    this.nextId = 1;        // 다음 할일 ID (자동 증가)
  }

  // 할일 추가
  add(text, priority = "medium") {
    const todo = {
      id: this.nextId++,    // ID를 부여하고 1 증가
      text,                 // text: text 의 축약형
      priority,
      done: false,
      createdAt: new Date()
    };
    this.todos.push(todo);
    console.log(`추가됨: "${text}" (ID: ${todo.id})`);
    return todo;
  }

  // 할일 완료 처리
  complete(id) {
    const todo = this.todos.find(t => t.id === id);
    if (todo) {
      todo.done = true;
      console.log(`완료: "${todo.text}"`);
    } else {
      console.log(`ID ${id}번 할일을 찾을 수 없습니다`);
    }
  }

  // 할일 삭제
  remove(id) {
    const index = this.todos.findIndex(t => t.id === id);
    if (index !== -1) {
      const removed = this.todos.splice(index, 1)[0];
      console.log(`삭제됨: "${removed.text}"`);
    }
  }

  // 미완료 할일만 보기
  getPending() {
    return this.todos.filter(t => !t.done);
  }

  // 전체 목록 출력
  list() {
    if (this.todos.length === 0) {
      console.log("할일이 없습니다!");
      return;
    }
    this.todos.forEach(t => {
```

```
      const status = t.done ? "[완료]" : "[    ]";
      console.log(`${status} #${t.id} ${t.text} (${t.priority})`);
    });
  }

  // static 메서드: 인스턴스 없이도 호출 가능
  static createFromArray(textArray) {
    const manager = new TodoManager();
    textArray.forEach(text => manager.add(text));
    return manager;
  }
}

// 사용 예제
const manager = new TodoManager();
manager.add("JavaScript 공부", "high");
manager.add("점심 먹기", "medium");
manager.add("이메일 확인", "low");

manager.list();
// [    ] #1 JavaScript 공부 (high)
// [    ] #2 점심 먹기 (medium)
// [    ] #3 이메일 확인 (low)

manager.complete(2);  // 완료: "점심 먹기"
manager.list();
// [    ] #1 JavaScript 공부 (high)
// [완료] #2 점심 먹기 (medium)
// [    ] #3 이메일 확인 (low)

// static 메서드 활용
const quickManager = TodoManager.createFromArray(["청소", "빨래", "요리"]);
quickManager.list();
```

3.6 JSON - 데이터 교환의 세계 공용어

왜 필요한가?

카카오톡에서 메시지를 보내면, 그 메시지가 **서버를 거쳐 상대방에게** 전달됩니다. 이때 메시지 데이터를 어떤 형식으로 보낼지 정해야 합니다. **JSON**은 이런 데이터 교환에 쓰이는 **세계 표준 형식**입니다.

```javascript
// JavaScript 객체
const user = {
  name: "김철수",
  age: 25,
  hobbies: ["독서", "코딩", "게임"]
};

// 객체 → JSON 문자열 (서버에 보낼 수 있는 형태로 변환)
const jsonString = JSON.stringify(user);
console.log(jsonString);
// '{"name":"김철수","age":25,"hobbies":["독서","코딩","게임"]}'
console.log(typeof jsonString);  // "string" (문자열!)

// JSON 문자열 → 객체 (서버에서 받은 데이터를 사용할 수 있게 변환)
const parsed = JSON.parse(jsonString);
console.log(parsed.name);       // "김철수"
console.log(parsed.hobbies[1]); // "코딩"

// 보기 좋게 출력 (들여쓰기 2칸)
console.log(JSON.stringify(user, null, 2));
// {
//   "name": "김철수",
//   "age": 25,
//   "hobbies": [
//     "독서",
//     "코딩",
//     "게임"
//   ]
// }
```

자주 하는 실수

```
// 실수 1: JSON에서는 작은따옴표 사용 불가
// '{"name": '김철수'}'  // 에러!
// '{"name": "김철수"}'  // 올바른 JSON

// 실수 2: JSON.parse에 잘못된 문자열 넣기
try {
  JSON.parse("이것은 JSON이 아닙니다");
} catch (error) {
  console.log("JSON 파싱 에러:", error.message);
}

// 실수 3: 함수는 JSON으로 변환되지 않는다
const obj = {
  name: "철수",
  greet() { return "안녕!"; }  // 함수
};
console.log(JSON.stringify(obj));  // {"name":"철수"} (함수는 사라진다!)
```

Day 04 - 비동기 프로그래밍 (서버와 통신하기)

4.1 동기 vs 비동기

실생활 비유: 카페 주문

동기 (Synchronous): 편의점 계산대

- 앞사람 계산이 끝날 때까지 줄 서서 기다려야 한다
- 한 번에 하나씩만 처리

비동기 (Asynchronous): 카페 주문

- 주문하고 번호표를 받는다
- 자리에 앉아서 다른 일을 한다 (책 읽기, 폰 보기)
- 번호가 호출되면 음료를 받으러 간다

```
// 동기 코드: 한 줄씩 순서대로 실행 (편의점 줄 서기)
console.log("1. 주문 시작");
console.log("2. 음료 만드는 중... (3분 걸림)");
console.log("3. 음료 완성!");
console.log("4. 다음 손님");
// 결과: 1 → 2 → 3 → 4 (순서대로)

// 비동기 코드: 기다리는 동안 다른 일 처리 (카페 번호표)
console.log("1. 주문 시작");
setTimeout(() => {
  // setTimeout: "3초 후에 이 코드를 실행해줘"
  console.log("3. 음료 완성! (3초 후)");
}, 3000);
console.log("2. 번호표 받고 자리에 앉음");
console.log("4. 핸드폰 보는 중...");
// 결과: 1 → 2 → 4 → (3초 후) 3
```

왜 비동기가 필요한가?

```
// 서버에서 데이터를 가져올 때
// 네트워크 요청은 0.1초 ~ 수 초가 걸릴 수 있다
// 이 시간 동안 화면이 멈추면 사용자가 답답해한다!

// 카카오톡을 떠올려보세요:
// - 메시지를 보내는 동안에도 다른 채팅방을 볼 수 있다
// - 사진을 업로드하는 동안에도 텍스트를 입력할 수 있다
// 이것이 바로 "비동기" 덕분입니다!
```

4.2 콜백 지옥 (Callback Hell)

```
// 콜백 중첩이 깊어지면... (실제로 이렇게 쓰면 안 됩니다!)
// 로그인 → 사용자 정보 가져오기 → 주문 내역 가져오기 → 결제하기
setTimeout(() => {
  console.log("1. 로그인 성공");
  setTimeout(() => {
    console.log("2. 사용자 정보 가져옴");
    setTimeout(() => {
      console.log("3. 주문 내역 가져옴");
      setTimeout(() => {
        console.log("4. 결제 완료!");
        // 점점 안쪽으로 들어가서 읽기 어렵다... (콜백 지옥!)
      }, 1000);
    }, 1000);
  }, 1000);
}, 1000);

// 해결책: Promise와 async/await (아래에서 설명)
```

4.3 Promise (약속)

실생활 비유: 배달 추적

Promise는 **배달 추적 시스템**과 같습니다.

- **pending (대기중)**: "라이더가 픽업했습니다" (아직 도착 안 함)
- **fulfilled (이행됨)**: "배달이 완료되었습니다!" (성공)
- **rejected (거부됨)**: "배달이 취소되었습니다" (실패)

```javascript
// Promise 만들기
function orderFood(menu) {
  return new Promise((resolve, reject) => {
    // resolve: 성공했을 때 호출하는 함수
    // reject: 실패했을 때 호출하는 함수

    console.log(`${menu} 주문 접수됨...`);

    setTimeout(() => {
      // 80% 확률로 성공, 20% 확률로 실패
      if (Math.random() > 0.2) {
        resolve(`${menu} 배달 완료!`);       // 성공!
      } else {
        reject(new Error(`${menu} 품절!`));  // 실패!
      }
    }, 2000);
  });
}

// Promise 사용하기: .then() .catch() .finally()
orderFood("치킨")
  .then((result) => {
    // 성공했을 때 실행 (배달 완료!)
    console.log(result);  // "치킨 배달 완료!"
  })
  .catch((error) => {
    // 실패했을 때 실행 (품절!)
    console.log("에러:", error.message);  // "에러: 치킨 품절!"
  })
  .finally(() => {
    // 성공이든 실패든 항상 실행
    console.log("주문 처리 종료");
  });
```

Promise 체이닝 (연결하기)

```javascript
// 체이닝으로 콜백 지옥 해결!
function login(userId) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("1. 로그인 성공");
      resolve({ userId, token: "abc123" });
    }, 1000);
  });
}

function getUserInfo(token) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("2. 사용자 정보 가져옴");
      resolve({ name: "김철수", email: "cs@test.com" });
    }, 1000);
  });
}

function getOrders(email) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("3. 주문 내역 가져옴");
      resolve(["주문1", "주문2", "주문3"]);
    }, 1000);
  });
}

// then 체이닝: 순서대로 실행되지만 깔끔하다!
login("chulsu")
  .then(user => getUserInfo(user.token))
  .then(info => getOrders(info.email))
  .then(orders => {
    console.log("주문 목록:", orders);
  })
  .catch(error => {
    console.log("어딘가에서 에러:", error.message);
  });
```

Promise.all - 여러 작업 동시에

```
// 여러 API를 동시에 호출할 때 (배달앱에서 여러 메뉴 동시 주문)
const promise1 = new Promise(resolve ⇒
  setTimeout(() ⇒ resolve("한식 도착!"), 2000)
);
const promise2 = new Promise(resolve ⇒
  setTimeout(() ⇒ resolve("중식 도착!"), 1500)
);
const promise3 = new Promise(resolve ⇒
  setTimeout(() ⇒ resolve("양식 도착!"), 3000)
);

// Promise.all: 모든 작업이 끝날 때까지 기다린다
Promise.all([promise1, promise2, promise3])
  .then(results ⇒ {
    console.log(results);
    // ["한식 도착!", "중식 도착!", "양식 도착!"]
    // 가장 오래 걸리는 3초 후에 모든 결과를 한번에 받음
  })
  .catch(error ⇒ {
    // 하나라도 실패하면 전체가 실패!
    console.log("에러:", error);
  });

// Promise.allSettled: 성공/실패 상관없이 모두 완료될 때까지 기다린다
const p1 = Promise.resolve("성공!");
const p2 = Promise.reject(new Error("실패!"));
const p3 = Promise.resolve("성공!");

Promise.allSettled([p1, p2, p3]).then(results ⇒ {
  results.forEach(result ⇒ {
    if (result.status ═══ "fulfilled") {
      console.log("성공:", result.value);
    } else {
      console.log("실패:", result.reason.message);
    }
  });
  // 성공: 성공!
  // 실패: 실패!
  // 성공: 성공!
});
```

4.4 async/await - 현대적 비동기

왜 필요한가?

`.then()` 체이닝도 좋지만, **동기 코드처럼 읽히는** `async/await`이 더 직관적입니다.

```
// .then() 방식 (Before)
function fetchDataThen() {
  login("chulsu")
    .then(user => getUserInfo(user.token))
    .then(info => getOrders(info.email))
    .then(orders => console.log(orders))
    .catch(error => console.log(error));
}

// async/await 방식 (After) - 훨씬 읽기 쉽다!
async function fetchDataAsync() {
  try {
    const user = await login("chulsu");          // 로그인 끝날 때까지 기다림
    const info = await getUserInfo(user.token);  // 정보 가져올 때까지 기다림
    const orders = await getOrders(info.email);  // 주문 가져올 때까지 기다림
    console.log(orders);
  } catch (error) {
    console.log("에러:", error.message);
  }
}
```

async/await 기본 사용법

```
// async: "이 함수 안에서 await을 사용할 거야"라고 선언
// await: "이 Promise가 끝날 때까지 기다려"라고 지시

async function getWeather(city) {
  try {
    console.log(`${city} 날씨 조회 중...`);

    // await은 Promise가 resolve될 때까지 기다린다
    const response = await new Promise((resolve) => {
      setTimeout(() => {
        resolve({ city, temp: 22, condition: "맑음" });
      }, 1000);
    });

    console.log(`${response.city}: ${response.temp}도, ${response.condition}`);
    return response;

  } catch (error) {
    // reject되면 catch로 들어온다
    console.log("날씨 조회 실패:", error.message);
  } finally {
    // 성공이든 실패든 항상 실행
    console.log("날씨 조회 종료");
  }
}

// async 함수 호출
getWeather("서울");
// 서울 날씨 조회 중...
// (1초 후)
// 서울: 22도, 맑음
// 날씨 조회 종료
```

4.5 에러 처리

try/catch/finally

```
// try: "이 코드를 시도해봐"
// catch: "에러가 나면 이렇게 처리해"
// finally: "에러 여부와 관계없이 이것은 꼭 해"

function divide(a, b) {
  if (b === 0) {
    throw new Error("0으로 나눌 수 없습니다!");  // 에러 발생!
  }
  return a / b;
}

try {
  console.log(divide(10, 2));   // 5
  console.log(divide(10, 0));   // 에러 발생! → catch로 이동
  console.log("이 줄은 실행되지 않음");
} catch (error) {
  console.log("에러 발생:", error.message);  // "0으로 나눌 수 없습니다!"
} finally {
  console.log("계산 종료");  // 항상 실행됨
}
```

커스텀 에러 클래스

```javascript
// 회원가입 유효성 검사용 커스텀 에러
class ValidationError extends Error {
  constructor(field, message) {
    super(message);           // 부모(Error) 생성자 호출
    this.name = "ValidationError";
    this.field = field;       // 어떤 필드에서 에러가 났는지
  }
}

function validateUser(user) {
  if (!user.name || user.name.trim() === "") {
    throw new ValidationError("name", "이름은 필수입니다");
  }
  if (!user.email || !user.email.includes("@")) {
    throw new ValidationError("email", "올바른 이메일 형식이 아닙니다");
  }
  if (!user.age || user.age < 0) {
    throw new ValidationError("age", "나이는 0 이상이어야 합니다");
  }
  return true;
}

try {
  validateUser({ name: "철수", email: "invalid-email", age: 25 });
} catch (error) {
  if (error instanceof ValidationError) {
    console.log(`[${error.field}] ${error.message}`);
    // [email] 올바른 이메일 형식이 아닙니다
  } else {
    console.log("알 수 없는 에러:", error.message);
  }
}
```

재시도(Retry) 패턴

```javascript
// 네트워크 요청이 실패하면 자동으로 다시 시도하는 패턴
async function fetchWithRetry(url, maxRetries = 3) {
  for (let attempt = 1; attempt ≤ maxRetries; attempt++) {
    try {
      console.log(`시도 ${attempt}/${maxRetries}...`);

      // 실제로는 fetch(url)을 쓰지만, 여기서는 시뮬레이션
      const result = await new Promise((resolve, reject) => {
        // 50% 확률로 성공
        if (Math.random() > 0.5) {
          resolve({ data: "성공 데이터!" });
        } else {
          reject(new Error("네트워크 오류"));
        }
      });

      console.log("성공!", result);
      return result;  // 성공하면 바로 반환

    } catch (error) {
      console.log(`시도 ${attempt} 실패: ${error.message}`);

      if (attempt === maxRetries) {
        throw new Error(`${maxRetries}번 시도 후 최종 실패`);
      }

      // 잠시 대기 후 재시도 (1초, 2초, 3초... 점점 길어짐)
      const waitTime = attempt * 1000;
      console.log(`${waitTime / 1000}초 후 재시도...`);
      await new Promise(r => setTimeout(r, waitTime));
    }
  }
}

// 사용
fetchWithRetry("https://api.example.com/data")
  .then(data => console.log("최종 결과:", data))
  .catch(error => console.log("모든 시도 실패:", error.message));
```

4.6 fetch API (맛보기)

```javascript
// fetch: 서버에서 데이터를 가져오는 브라우저 내장 함수
// 실제 API를 사용하는 기본 패턴

async function fetchUsers() {
  try {
    // 1단계: 서버에 요청 보내기
    const response = await fetch("https://jsonplaceholder.typicode.com/users");

    // 2단계: 응답이 정상인지 확인
    if (!response.ok) {
      throw new Error(`HTTP 에러! 상태 코드: ${response.status}`);
    }

    // 3단계: JSON 데이터로 변환
    const users = await response.json();

    // 4단계: 데이터 사용
    users.forEach(user => {
      console.log(`${user.name} (${user.email})`);
    });

    return users;

  } catch (error) {
    console.log("데이터 가져오기 실패:", error.message);
  }
}

// fetchUsers();  // 실행하면 실제 사용자 데이터를 가져옴
```

4.7 실습: 비동기 할일 저장/불러오기

```javascript
// === 비동기 할일 저장 시스템 (시뮬레이션) ===

// 가짜 데이터베이스 (실제로는 서버에 저장)
let fakeDB = [];

// 할일 저장 (서버에 보내는 것을 시뮬레이션)
function saveTodo(todo) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (!todo.text || todo.text.trim() === "") {
        reject(new Error("할일 내용이 비어있습니다"));
        return;
      }
      const saved = { ...todo, id: Date.now(), savedAt: new Date() };
      fakeDB.push(saved);
      resolve(saved);
    }, 500);  // 네트워크 지연 시뮬레이션
  });
}

// 할일 목록 불러오기
function loadTodos() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve([...fakeDB]);  // 복사본 반환
    }, 300);
  });
}

// 할일 삭제
function deleteTodo(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const index = fakeDB.findIndex(t => t.id === id);
      if (index === -1) {
        reject(new Error("해당 할일을 찾을 수 없습니다"));
        return;
      }
      const deleted = fakeDB.splice(index, 1)[0];
      resolve(deleted);
    }, 300);
  });
}

// async/await으로 사용하기
async function main() {
  try {
    // 할일 추가
    console.log("--- 할일 추가 ---");
    const todo1 = await saveTodo({ text: "JavaScript 공부", priority: "high" });
    console.log("저장됨:", todo1);
```

```
    const todo2 = await saveTodo({ text: "점심 먹기", priority: "medium" });
    console.log("저장됨:", todo2);

    const todo3 = await saveTodo({ text: "운동하기", priority: "low" });
    console.log("저장됨:", todo3);

    // 전체 목록 조회
    console.log("\n--- 전체 목록 ---");
    const allTodos = await loadTodos();
    allTodos.forEach(t => console.log(`  [${t.priority}] ${t.text}`));

    // 하나 삭제
    console.log("\n--- 삭제 ---");
    const deleted = await deleteTodo(todo2.id);
    console.log("삭제됨:", deleted.text);

    // 삭제 후 목록
    console.log("\n--- 삭제 후 목록 ---");
    const remaining = await loadTodos();
    remaining.forEach(t => console.log(`  [${t.priority}] ${t.text}`));

    // 빈 할일 저장 시도 (에러 테스트)
    console.log("\n--- 에러 테스트 ---");
    await saveTodo({ text: "" });

  } catch (error) {
    console.log("에러 발생:", error.message);
  }
}

main();
```

Day 05 - DOM과 이벤트 (브라우저에서 동작하기)

5.1 DOM이란?

실생활 비유: 가족 관계도

DOM(Document Object Model)은 HTML 문서를 **가족 관계도(트리 구조)**로 표현한 것입니다.

```
        document (시조)
            |
          html (할아버지)
         /        \
      head        body (부모)
       |         /    |     \
     title     h1   div     footer (자식)
                   / \
                  p   button (손자)
```

JavaScript는 이 가족 관계도를 통해 HTML 요소를 찾고, 바꾸고, 추가하고, 삭제할 수 있습니다.

```html
<!-- 이런 HTML이 있다면... -->
<div id="app">
  <h1 class="title">할일 목록</h1>
  <ul id="todo-list">
    <li class="todo-item">장보기</li>
    <li class="todo-item">공부하기</li>
  </ul>
  <button id="add-btn">추가</button>
</div>
```

왜 필요한가?

DOM이 없으면 JavaScript로 웹 페이지를 변경할 수 없습니다. 사용자가 버튼을 클릭했을 때 화면이 바뀌거나, 새로운 내용이 추가되는 등의 **동적인 웹 페이지**를 만들려면 DOM 조작이 필수입니다.

5.2 DOM 요소 선택

```
// 1. getElementById: ID로 하나의 요소 찾기 (주민등록번호처럼 유일)
const addBtn = document.getElementById("add-btn");
console.log(addBtn);  // <button id="add-btn">추가</button>

// 2. querySelector: CSS 선택자로 첫 번째 요소 찾기 (가장 많이 사용!)
const title = document.querySelector(".title");      // 클래스로 찾기
const firstItem = document.querySelector("li");       // 태그로 찾기
const app = document.querySelector("#app");            // ID로 찾기

// 3. querySelectorAll: CSS 선택자로 모든 요소 찾기 (여러 개!)
const allItems = document.querySelectorAll(".todo-item");
console.log(allItems.length);  // 2

// querySelectorAll은 NodeList를 반환 (배열과 비슷)
allItems.forEach(item => {
  console.log(item.textContent);  // "장보기", "공부하기"
});

// CSS 선택자를 활용한 다양한 찾기
document.querySelector("div > h1");          // div의 직속 자식 h1
document.querySelector("ul li:first-child"); // ul의 첫 번째 li
document.querySelector("[data-id='1']");      // data-id 속성이 1인 요소
```

5.3 DOM 조작

텍스트 변경

```
// textContent: 요소의 텍스트만 변경 (안전)
const title = document.querySelector(".title");
title.textContent = "나의 할일 목록";

// innerHTML: HTML 태그도 포함해서 변경 (주의 필요!)
const container = document.querySelector("#app");
// 주의: 사용자 입력을 innerHTML에 직접 넣으면 보안 위험 (XSS 공격)
container.innerHTML = "<h1>새로운 제목</h1><p>새로운 내용</p>";
```

스타일 변경

```
const title = document.querySelector(".title");

// style 속성으로 직접 변경 (CSS 속성명이 camelCase로 바뀜)
title.style.color = "blue";               // color: blue;
title.style.fontSize = "24px";            // font-size: 24px;
title.style.backgroundColor = "#f0f0f0"; // background-color: #f0f0f0;

// classList: 클래스 추가/제거 (더 좋은 방법!)
const item = document.querySelector(".todo-item");
item.classList.add("completed");       // 클래스 추가
item.classList.remove("completed");    // 클래스 제거
item.classList.toggle("completed");    // 있으면 제거, 없으면 추가
item.classList.contains("completed"); // 클래스가 있는지 확인 (true/false)
```

요소 생성과 추가

```
// 새 요소 만들기
const newItem = document.createElement("li");      // <li> 태그 생성
newItem.textContent = "새 할일";                    // 텍스트 설정
newItem.className = "todo-item";                     // 클래스 설정
newItem.setAttribute("data-id", "3");               // 속성 설정

// 요소 추가하기
const todoList = document.querySelector("#todo-list");
todoList.appendChild(newItem);  // 목록 끝에 추가

// 특정 위치에 추가
const firstChild = todoList.firstElementChild;
todoList.insertBefore(newItem, firstChild);  // 맨 앞에 추가

// 요소 삭제
const oldItem = document.querySelector(".todo-item");
oldItem.remove();  // 자기 자신 삭제

// 부모를 통한 삭제
// todoList.removeChild(oldItem);  // 부모에서 자식 제거
```

5.4 이벤트 처리

addEventListener 기본 사용법

```
// "이 요소에서 이런 일이 일어나면, 이 함수를 실행해줘"
const button = document.querySelector("#add-btn");

// 클릭 이벤트 등록
button.addEventListener("click", function(event) {
  // event 객체: 이벤트에 대한 상세 정보가 담겨있다
  console.log("버튼이 클릭됨!");
  console.log("클릭 위치:", event.clientX, event.clientY);
  console.log("클릭된 요소:", event.target);
});

// 화살표 함수로 더 간결하게
button.addEventListener("click", (e) => {
  console.log("클릭!");
});
```

자주 쓰는 이벤트

```javascript
// 1. click: 클릭했을 때
button.addEventListener("click", () => {
  console.log("클릭됨");
});

// 2. input: 입력 값이 변할 때 (실시간)
const inputField = document.querySelector("#todo-input");
inputField.addEventListener("input", (e) => {
  console.log("현재 입력:", e.target.value);
  // 실시간 검색, 글자 수 표시 등에 활용
});

// 3. submit: 폼이 제출될 때
const form = document.querySelector("#todo-form");
form.addEventListener("submit", (e) => {
  e.preventDefault();  // 중요! 페이지 새로고침 방지
  const value = inputField.value;
  console.log("제출된 값:", value);
});

// 4. keydown: 키보드 키를 눌렀을 때
inputField.addEventListener("keydown", (e) => {
  if (e.key === "Enter") {
    console.log("엔터 키 입력!");
  }
  if (e.key === "Escape") {
    console.log("ESC 키 입력! 취소");
    inputField.value = "";  // 입력 필드 비우기
  }
});
```

event.preventDefault() - 기본 동작 방지

```javascript
// 폼 제출 시 페이지가 새로고침되는 기본 동작을 막아야 한다!
const form = document.querySelector("form");

// 방지 안 하면: 폼 제출 → 페이지 새로고침 → JavaScript 상태 초기화!
// 방지하면: 폼 제출 → JavaScript로 데이터 처리 → 페이지 유지

form.addEventListener("submit", (e) => {
  e.preventDefault();  // "기본 동작 하지 마!"

  // 이제 JavaScript로 자유롭게 처리
  const formData = new FormData(form);
  console.log("입력된 데이터:", formData.get("todoText"));
});
```

5.5 이벤트 버블링

실생활 비유: 물속 거품

물속에서 거품(이벤트)은 **아래에서 위로 올라갑니다**. DOM에서도 이벤트는 클릭된 요소에서 시작해 **부모 → 조부모 → ... → document** 순으로 올라갑니다.

```html
<!— HTML 구조 -->
<div id="outer" style="padding: 20px; background: lightblue;">
  외부 div
  <div id="inner" style="padding: 20px; background: lightgreen;">
    내부 div
    <button id="btn">클릭!</button>
  </div>
</div>
```

```javascript
// 이벤트 버블링: 안쪽에서 바깥쪽으로 전파
document.querySelector("#btn").addEventListener("click", () => {
  console.log("1. 버튼 클릭!");
});

document.querySelector("#inner").addEventListener("click", () => {
  console.log("2. 내부 div 클릭!");
});

document.querySelector("#outer").addEventListener("click", () => {
  console.log("3. 외부 div 클릭!");
});

// 버튼을 클릭하면?
// 1. 버튼 클릭!      (자기 자신)
// 2. 내부 div 클릭!  (부모)
// 3. 외부 div 클릭!  (조부모)
// → 이벤트가 거품처럼 위로 올라간다!

// stopPropagation: 버블링 중단
document.querySelector("#btn").addEventListener("click", (e) => {
  e.stopPropagation();  // "여기서 멈춰! 더 이상 올라가지 마!"
  console.log("버튼만 클릭!");
});
// 이제 버튼을 클릭해도 부모/조부모 이벤트가 실행되지 않는다
```

5.6 이벤트 위임 (Event Delegation)

왜 필요한가?

할일 목록에 100개의 항목이 있다면, 각각에 이벤트를 거는 것은 비효율적입니다. 또한 나중에 추가되는 항목에는 이벤트가 걸리지 않는 문제가 있습니다.

```
// 나쁜 방법: 각 항목마다 이벤트 등록
// 항목이 100개면 이벤트 리스너도 100개! (메모리 낭비)
// 나중에 추가된 항목에는 이벤트가 없음!
document.querySelectorAll(".todo-item").forEach(item => {
  item.addEventListener("click", () => {
    item.classList.toggle("completed");
  });
});

// 좋은 방법: 부모에 한 번만 등록 (이벤트 위임)
// 이벤트 버블링을 이용해 부모에서 자식의 이벤트를 처리
const todoList = document.querySelector("#todo-list");

todoList.addEventListener("click", (e) => {
  // event.target: 실제로 클릭된 요소
  // closest(): 가장 가까운 조상 요소 찾기

  const item = e.target.closest(".todo-item");
  if (!item) return;  // .todo-item이 아닌 곳을 클릭하면 무시

  item.classList.toggle("completed");
  console.log("토글됨:", item.textContent);
});

// 이제 나중에 추가된 항목도 자동으로 이벤트가 동작한다!
```

event.target과 closest() 활용

```
// 할일 항목에 삭제 버튼이 있는 경우
// <li class="todo-item" data-id="1">
//   <span>장보기</span>
//   <button class="delete-btn">삭제</button>
// </li>

todoList.addEventListener("click", (e) => {
  // 삭제 버튼을 클릭한 경우
  if (e.target.classList.contains("delete-btn")) {
    const item = e.target.closest(".todo-item");
    const id = item.dataset.id;  // data-id 속성 값 가져오기
    console.log(`ID ${id} 삭제!`);
    item.remove();  // DOM에서 제거
    return;
  }

  // 할일 항목 자체를 클릭한 경우 (완료 토글)
  const item = e.target.closest(".todo-item");
  if (item) {
    item.classList.toggle("completed");
  }
});
```

5.7 실습: 브라우저 할일 앱 완성

아래는 HTML, CSS, JavaScript를 모두 포함한 완전한 할일 앱입니다.

HTML 구조

```html
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>나의 할일 앱</title>
  <style>
    /* 간단한 스타일 */
    body { font-family: 'Noto Sans KR', sans-serif; max-width: 500px; margin: 50p
    h1 { color: #2b6cb0; text-align: center; }
    #todo-form { display: flex; gap: 8px; margin-bottom: 20px; }
    #todo-input { flex: 1; padding: 10px; font-size: 16px; border: 2px solid #ccc
    #todo-input:focus { border-color: #2b6cb0; outline: none; }
    button { padding: 10px 20px; background: #2b6cb0; color: white; border: none;
    button:hover { background: #1a4a8a; }
    .todo-item { display: flex; align-items: center; padding: 12px; border-bottom
    .todo-item.completed span { text-decoration: line-through; color: #999; }
    .todo-item span { flex: 1; cursor: pointer; }
    .delete-btn { background: #e53e3e; padding: 4px 10px; font-size: 12px; }
    .delete-btn:hover { background: #c53030; }
    .stats { text-align: center; color: #666; margin-top: 16px; }
    .filter-bar { display: flex; gap: 8px; justify-content: center; margin-bottom
    .filter-bar button { padding: 6px 14px; font-size: 13px; background: #eee; co
    .filter-bar button.active { background: #2b6cb0; color: white; }
  </style>
</head>
<body>
  <h1>나의 할일 앱</h1>

  <!— 할일 입력 폼 -->
  <form id="todo-form">
    <input type="text" id="todo-input" placeholder="할일을 입력하세요..." />
    <button type="submit">추가</button>
  </form>

  <!— 필터 버튼 -->
  <div class="filter-bar">
    <button class="filter-btn active" data-filter="all">전체</button>
    <button class="filter-btn" data-filter="pending">미완료</button>
    <button class="filter-btn" data-filter="completed">완료</button>
  </div>

  <!— 할일 목록 -->
  <div id="todo-list"></div>

  <!— 통계 -->
  <div class="stats" id="stats"></div>

  <script>
    // ===== JavaScript: 할일 앱 로직 =====

    // --- 상태 관리 ---
```

```javascript
let todos = [];           // 할일 배열
let currentFilter = "all"; // 현재 필터 상태
let nextId = 1;           // 다음 할일 ID

// --- DOM 요소 선택 ---
const form = document.querySelector("#todo-form");
const input = document.querySelector("#todo-input");
const todoListEl = document.querySelector("#todo-list");
const statsEl = document.querySelector("#stats");
const filterBar = document.querySelector(".filter-bar");

// --- 할일 추가 ---
function addTodo(text) {
  const todo = {
    id: nextId++,
    text: text.trim(),
    done: false,
    createdAt: new Date()
  };
  todos.push(todo);
  render();    // 화면 업데이트
}

// --- 할일 토글 (완료/미완료) ---
function toggleTodo(id) {
  const todo = todos.find(t => t.id === id);
  if (todo) {
    todo.done = !todo.done;
    render();
  }
}

// --- 할일 삭제 ---
function deleteTodo(id) {
  todos = todos.filter(t => t.id !== id);
  render();
}

// --- 필터링된 할일 가져오기 ---
function getFilteredTodos() {
  switch (currentFilter) {
    case "pending":
      return todos.filter(t => !t.done);
    case "completed":
      return todos.filter(t => t.done);
    default:
      return todos;
  }
}

// --- 화면 렌더링 ---
function render() {
  const filtered = getFilteredTodos();
```

```
  // 할일 목록 HTML 생성
  todoListEl.innerHTML = filtered.map(todo => `
    <div class="todo-item ${todo.done ? 'completed' : ''}" data-id="${todo.id
      <span>${todo.text}</span>
      <button class="delete-btn">삭제</button>
    </div>
  `).join("");

  // 통계 업데이트
  const total = todos.length;
  const completed = todos.filter(t => t.done).length;
  statsEl.textContent = `전체: ${total}개 | 완료: ${completed}개 | 남음: ${total
}

// --- 이벤트 등록 ---

// 폼 제출 이벤트
form.addEventListener("submit", (e) => {
  e.preventDefault();                 // 페이지 새로고침 방지!
  const text = input.value.trim();
  if (text === "") return;            // 빈 입력 무시
  addTodo(text);
  input.value = "";                   // 입력 필드 비우기
  input.focus();                      // 입력 필드에 포커스
});

// 할일 목록 이벤트 위임 (클릭)
todoListEl.addEventListener("click", (e) => {
  const item = e.target.closest(".todo-item");
  if (!item) return;

  const id = Number(item.dataset.id);

  // 삭제 버튼 클릭
  if (e.target.classList.contains("delete-btn")) {
    deleteTodo(id);
    return;
  }

  // 할일 텍스트 클릭 (토글)
  toggleTodo(id);
});

// 필터 버튼 이벤트 위임
filterBar.addEventListener("click", (e) => {
  if (!e.target.classList.contains("filter-btn")) return;

  // 활성 버튼 스타일 변경
  document.querySelectorAll(".filter-btn").forEach(btn => {
    btn.classList.remove("active");
  });
  e.target.classList.add("active");

  // 필터 변경 및 렌더링
```

```
      currentFilter = e.target.dataset.filter;
      render();
    });

    // --- 초기 렌더링 ---
    render();
  </script>
</body>
</html>
```

> **이 파일을 `todo-app.html` 로 저장하고 브라우저에서 열면 바로 동작합니다!**

Day 05 마무리: 주요 개념 정리

개념	설명	핵심 메서드
DOM 선택	HTML 요소 찾기	`querySelector`, `querySelectorAll`
DOM 조작	요소 변경/생성/삭제	`textContent`, `classList`, `createElement`
이벤트 등록	사용자 동작에 반응	`addEventListener`
이벤트 버블링	이벤트가 부모로 전파	`stopPropagation`
이벤트 위임	부모에서 자식 이벤트 처리	`event.target`, `closest()`

Phase 1 전체 요약

지금까지 5일 동안 JavaScript의 핵심을 배웠습니다:

- **Day 01**: 변수, 타입, 문자열, 배열 - 데이터를 담고 다루는 기초
- **Day 02**: 함수와 스코프 - 코드를 재사용하고 구조화하는 방법
- **Day 03**: 객체와 클래스 - 관련 데이터를 하나로 묶어 관리하는 방법
- **Day 04**: 비동기 프로그래밍 - 서버 통신과 시간이 걸리는 작업 처리
- **Day 05**: DOM과 이벤트 - 브라우저에서 사용자와 상호작용하는 방법

> **다음 단계:** Phase 2에서는 TypeScript를 배워 더 안전한 코드를 작성하는 방법을 학습합니다.